



Digital Kakeibo

10-Phase Build Plan

From Empty Repo to Launched Product



A journey of a thousand miles begins with a single step.

How to Use This Plan

This document breaks the Digital Kakeibo build into 10 sequential phases. Each phase is a self-contained chunk of work that produces something testable. Don't move to the next phase until the current one is genuinely done — finishing strong matters more than rushing.

What's in each phase

Goal: the one thing this phase exists to accomplish

Features built: the user-facing or technical capabilities delivered

Technical deep-dive: specific implementation details, libraries, code patterns

Acceptance criteria: concrete checks that prove the phase is done

Common pitfalls: things that will trip her up and how to avoid them

Suggested timeline

Aim for ~6 days per phase = roughly 8-10 weeks total. Phases 5 and 6 are the heaviest — give them more time. Phase 10 is the most important — never skip it.

What you're building

Before writing a single feature, set up the scaffolding properly. This phase is about boring-but-critical work that makes every later phase faster. Skip this phase's discipline and you'll pay for it tenfold later.

Features built

Working monorepo with Next.js frontend + Express backend

MongoDB Atlas connection (free tier is fine)

Sign up, login, logout flows — actually functional

Protected routes (logged-in users only)

Basic landing page (placeholder, will redesign in Phase 9)

Deployed to Vercel + Render with custom domain

Technical deep-dive

Project structure

digital-kakeibo/

■■■ client/ (Next.js 14, App Router)

■ ■■■ app/

■ ■■■ components/

■ ■ ■ ■ lib/ (api client, utils)

■ ■ ■ ■ styles/

■ ■ ■ ■ server/ (Express + Mongoose)

■ ■ ■ ■ routes/

■ ■ ■ ■ models/

■ ■ ■ ■ middleware/

■ ■ ■ ■ controllers/

■ ■ ■ ■ shared/ (TypeScript types)

Auth implementation

Use JWT with refresh token pattern. Access token (15 min expiry) stored in memory; refresh token (7 days) in httpOnly cookie. Refresh endpoint rotates both. This is the pattern Japanese fintechs use — interviewers will recognize it.

Key endpoints to build

POST /api/auth/signup { email, password } → { user, accessToken }

POST /api/auth/login { email, password } → { user, accessToken }

POST /api/auth/refresh (cookie) → { accessToken }

POST /api/auth/logout → 200

GET /api/auth/me (auth) → { user }

Password hashing

Use bcrypt with 12 rounds. Never store plaintext. Never log passwords. Add a check that rejects passwords under 8 characters with at least one number.

MongoDB schema (User only for now)

```
// server/models/User.js
```

```
const userSchema = new Schema({
  email: { type: String, unique: true, required: true, lowercase: true },
  passwordHash: { type: String, required: true },
  locale: { type: String, enum: ['en','ja'], default: 'en' },
  createdAt: { type: Date, default: Date.now },
  settings: {
    weeklyReminderDay: { type: Number, default: 0 }, // 0=Sun
    currency: { type: String, default: 'JPY' }
  }
});
```

Frontend auth state

Use React Context for auth state — simple and sufficient. Don't reach for Redux. Wrap protected routes in a <ProtectedRoute /> component that redirects to /login if no token.

Acceptance criteria

New user can sign up with email + password

Can log in, get redirected to /dashboard (placeholder page)

Refreshing the page keeps user logged in

Logout actually clears session

Protected route /dashboard redirects to /login when not authenticated

App is live on a real URL (e.g., `digitalkakeibo.app`)

Common pitfalls

CORS issues: set `credentials: true` on both frontend `fetch` and backend `cors()` — both are needed for refresh cookies to work

Environment variables: use `.env.example` committed to git, `.env` in gitignore. Document every variable.

Don't deploy until auth works locally: deploying broken code wastes hours debugging environment issues that aren't actually environment issues

What you're building

This phase is unusual in build plans — most devs jump straight to features and design as they go. For this product, that's fatal. The aesthetic IS the product. Spend a week getting the design system right, and every later phase becomes easier.

Features built

Tailwind config with custom design tokens

Reusable component library: Button, Input, Card, Modal, Toast

Typography system (Japanese serif + clean sans-serif)

Color palette implemented as CSS variables

Storybook setup for visual reference (optional but recommended)

Light/dark mode toggle (dark mode uses ink-on-paper aesthetic)

Technical deep-dive

Design tokens

```
// tailwind.config.js
```

```
colors: {
```

```
  sumi: '#1A1A1A', // ink black
```

```
  kinari: '#F5F0E8', // unbleached cream
```

```
  enji: '#B8454D', // crimson accent
```

```
  ai: '#2E4756', // indigo (secondary)
```

```
  cha: '#8B6F47', // tea brown (tertiary)
```

```
  washi: '#FAF7F0', // paper white
```

```
}
```

Typography

Use Google Fonts:

Headings: Shippori Mincho B1 (Japanese-influenced serif, free)

Body: Inter (clean, highly readable)

Numerals: JetBrains Mono for amounts (gives money the gravity it deserves)

Spacing philosophy

Default spacing should feel generous. Replace Tailwind's default scale with one tuned for breathing room — minimum padding is `p-6`, not `p-2`. Build a ``<Spacer />`` component used liberally.

Component patterns

Build these in this order: Button → Input → Card → Modal → Toast. Each component has a single visual style — no variants. Variants come later when you actually need them.

Button example

// Two states only: default and loading

// One color: black on cream, with subtle hover

// No icons in MVP — text only

`<Button onClick={save}>■■■■ Record</Button>`

Acceptance criteria

Open any component in isolation and it looks polished

Color palette is in CSS variables, not hardcoded anywhere

Every text element uses defined typography tokens

A throwaway test page using these components looks better than 90% of finance apps

Dark mode works (even if she only uses light mode personally)

Common pitfalls

Resist adding component variants you don't need yet

Don't use shadcn/ui — it'll override your aesthetic with its own

Test components on a real iPhone, not just desktop browser

What you're building

This is the most-used feature in the app. Users will hit this flow 5-15 times per week. Every detail matters. Spend time getting the feel right — the small pause after save, the way the amount renders, the gentle confirmation.

Features built

Add Entry screen (full-screen, distraction-free)

Four-category selection UI (large tap targets, Japanese + English labels)

Optional one-line note field (50 character soft limit)

Entries list view (chronological, grouped by day)

Edit entry (with confirmation)

Delete entry (with confirmation + undo toast)

"Today's entries" widget on dashboard

Technical deep-dive

Entry schema

```
// server/models/Entry.js

const entrySchema = new Schema({
  userId: { type: ObjectId, ref: 'User', required: true, index: true },
  amount: { type: Number, required: true, min: 0 },
  currency: { type: String, default: 'JPY' },
  category: {
    type: String,
    enum: ['needs', 'wants', 'culture', 'unexpected'],
    required: true
  },
  note: { type: String, maxLength: 200 },
  date: { type: Date, required: true, default: Date.now },
  createdAt: { type: Date, default: Date.now }
});

entrySchema.index({ userId: 1, date: -1 });
```

API endpoints

POST /api/entries { amount, category, note?, date? }

GET /api/entries ?from=&to=&category=

GET /api/entries/:id

PATCH /api/entries/:id

DELETE /api/entries/:id

The mindful entry UX

The flow has three deliberate moments:

Tap '+' opens a full-screen sheet (not a modal). The amount field is auto-focused. As digits are typed, they appear in large brushstroke-style typography centered on screen.

Below the amount, four category buttons. They're large — minimum 60pt height. Tapping selects (with subtle haptic if mobile). The selected category gets a thin black underline, ink-style.

After save, the screen doesn't immediately dismiss. There's a 600ms moment where the entry fades down beneath a thin sumi line, with the words "■■■■■■■" (recorded) appearing softly. Then it dismisses.

State management

Use TanStack Query (React Query) for entries. It handles caching, optimistic updates, refetching — saves you a week of manual state work.

```
const { data: entries } = useQuery({
  queryKey: ['entries', { from, to }],
  queryFn: () => api.getEntries({ from, to })
});

const createEntry = useMutation({
  mutationFn: api.createEntry,
  onMutate: async (newEntry) => {
    // Optimistic update — entry appears instantly
  }
});
```

Acceptance criteria

Adding an entry takes ≤ 3 taps from any screen

Entry appears in the list within 100ms of save (optimistic)

List groups by day with Japanese-style date headers (e.g., "5月2日 ■■■")

Editing or deleting works with proper confirmations

The "after save" pause feels intentional, not laggy

Currency formatting respects user's locale

Common pitfalls

Don't allow custom categories: users will ask for them. The four-category constraint is a feature, not a limitation.

Don't add receipt scanning: tempting, but breaks the mindfulness principle. P2 at earliest.

Handle JPY vs other currencies: JPY has no decimals. ¥1000, not ¥1000.00. Format properly.

Timezones: store dates as UTC, display in user's local timezone. Use date-fns-tz.

What you're building

The home screen of the app. Most users will see this 30 seconds at a time, multiple times a day. Make those 30 seconds feel like opening a beautifully bound paper journal.

Features built

Dashboard layout with current week summary

Category bar visualization (the four categories, this week's spending)

Quick-add floating button

Recent entries (last 5)

Current month progress vs. savings goal (placeholder for now)

Empty state — beautiful, instructive, never pushy

Technical deep-dive

The category bar visualization

Skip the pie chart. Pie charts are clinical. Build a horizontal stacked bar — four segments, one per category, width proportional to spending. Each segment labeled with category name and amount. Segments use the four palette colors (sumi, ai, cha, enji at varying opacities).

Why horizontal bar over pie

Easier to compare proportions at a glance

Works on narrow mobile screens

Aesthetically calmer — no spinning angles

Aggregation query

```
// server/controllers/summary.js
```

```
async function getWeekSummary(userId, weekStart, weekEnd) {
```

```
  return Entry.aggregate([
```

```
    { $match: {
```

```
      userId,
```

```
      date: { $gte: weekStart, $lt: weekEnd }
```

```
    }},
```

```
    { $group: {
```

```
      _id: '$category',
```

```
      total: { $sum: '$amount' },
```

```
      count: { $sum: 1 }
```

```
    } }
```

```
  ]);
```

```
}
```

Empty state design

A new user sees the dashboard before any entries. Don't show empty charts — show a single elegant message:

Routing structure

```
/ (landing — public)
```

```
/login (public)
```

```
/signup (public)
```

```
/dashboard (auth — main app)
```

```
/entries (full entry list)
```

```
/entries/new (add entry sheet)
```

```
/entries/:id/edit (edit entry)
```

Acceptance criteria

Dashboard loads in under 1 second on 4G

Category bar updates in real-time when an entry is added

Empty state looks intentional, not unfinished

Mobile responsive — works on iPhone SE (smallest common screen)

Week boundaries respect user's timezone

Common pitfalls

Don't add too many widgets to the dashboard — restraint is the brand

Don't use Chart.js for the category bar — it's overkill. Build with CSS.

Test on slow networks — the aesthetic should still feel calm during loading

What you're building

This is where Digital Kakeibo stops being a budget app and becomes something deeper. The weekly reflection is the heart of the practice. It's also the metric that determines whether your product actually works.

Features built

Weekly reflection screen (Sundays by default, configurable)

Reflection prompt rotation (5+ prompts, randomly shown)

Reflection saving + journal storage

Past reflections browse view (chronological journal)

Email reminder system (single weekly email, gentle tone)

In-app notification (subtle, on Sunday evening)

Technical deep-dive

Reflection schema

```
// server/models/Reflection.js
```

```
const reflectionSchema = new Schema({
  userId: { type: ObjectId, ref: 'User', required: true, index: true },
  type: { type: String, enum: ['weekly', 'monthly'], required: true },
  period: { type: String, required: true }, // '2026-W18' or '2026-05'
  prompt: { type: String, required: true },
  response: { type: String, required: true, maxLength: 5000 },
  summarySnapshot: {
    totalSpent: Number,
    byCategory: { needs: Number, wants: Number, culture: Number, unexpected: Number }
  },
  createdAt: { type: Date, default: Date.now }
```



```
});
```

```
// One reflection per period
```

```
reflectionSchema.index({ userId: 1, type: 1, period: 1 }, { unique: true });
```

Prompt rotation

The same prompt every week gets stale. Build an array of 5-7 weekly prompts that rotate:

"What surprised you this week?"

"Which expense brought you the most joy?"

"Was there something you bought that you regret?"

"What did you NOT spend on that you're grateful for?"

"Was there a moment you almost spent and chose not to? What stopped you?"

"How did your spending match your values this week?"

Reminder system

Use a cron job (node-cron) running on Render. Every Sunday at 7pm in user's timezone, check who hasn't reflected on the past week, send a single email.

```
// server/jobs/weeklyReminder.js
```

```
cron.schedule('0 * * * *', async () => {
```

```
// Run hourly, find users where it's currently 7pm Sunday in their tz
```

```
const users = await User.find({...});
```

```
for (const user of users) {
```

```
const hasReflected = await Reflection.findOne({
```

```
userId: user._id, type: 'weekly', period: currentWeek
```

```
});
```

```
if (!hasReflected) await sendReminderEmail(user);
```

```
}
```

```
});
```

Email service

Use Resend (free tier: 3,000 emails/month). Cleaner API than SendGrid. The reminder email itself should match the brand — minimal HTML, no marketing styling.

The reflection UI

Full-screen, single column. Top: this week's category bar (the same one from dashboard). Middle: the prompt question in serif typography. Bottom: a single text area, no character count visible (it would create pressure). Save button at the very bottom.

Acceptance criteria

User can complete a reflection in under 2 minutes

Email reminder arrives at 7pm Sunday in user's timezone

Past reflections are browseable, beautifully formatted

```
const goalSchema = new Schema({
  userId: { type: ObjectId, ref: 'User', required: true },
```

```
monthYear: { type: String, required: true }, // '2026-05'
savingsTarget: { type: Number, required: true },
achieved: { type: Number, default: 0 },
reflectionId: { type: ObjectId, ref: 'Reflection' },
createdAt: { type: Date, default: Date.now }
});
goalSchema.index({ userId: 1, monthYear: 1 }, { unique: true });
```

Savings achievement calculation

This is subtle. "Savings achieved" = (income - total spent), where income comes from a separate question. For now, keep it simple: ask user to enter their income for the month at goal-setting time. Don't try to detect it automatically.

Monthly summary view

This becomes the "page" of the user's kakeibo book. Layout:

Header: month name in beautiful typography (e.g., "■■■ / May 2026")

Total spent — large numerals

Category breakdown (horizontal bar from Phase 4)

Savings: target vs. actual, with subtle visual

All four reflection answers, beautifully laid out

Footer: "■■■ " + completion date

Triggering the monthly ritual

On the first time the user opens the app on or after the 1st of the month, show a modal:

Allow skip. Don't be aggressive. The user can find the monthly ritual later from the menu.

Acceptance criteria

Four questions displayed sequentially, full-screen, beautiful

Monthly summary page is visually striking — looks worth printing

Savings goal is set for the upcoming month after closing the previous

Past months archive is browseable in reverse chronological order

Cannot complete the same month's ritual twice

Skipping the prompt is gracefully handled (offered again 3 days later)

Common pitfalls

Don't put progress dots like "1 of 4" — it makes the questions feel transactional

Don't auto-save partial answers — questions are answered together as a ritual

Test the date logic carefully — months have different lengths, timezones complicate things

Save the user's locale's question text in the reflection record (so future locale changes don't rewrite history)

What you're building

This phase is where growth begins. Every shared card is an ad. The card must be objectively beautiful — pretty enough that even non-users want to know what app made it. Spend two days on the visual design before writing any code.

Features built

"Share" button on monthly summary page

Generated card image (1080x1920 for Instagram Stories)

Card customization options (light/dark, with/without amounts)

Native share API integration (iOS/Android)

Shareable card landing page (/share/:cardId for Twitter unfurls)

Subtle watermark with app name ("digitalkakeibo.app")

Technical deep-dive

Card design

Vertical layout, 1080x1920 (Instagram Story dimensions, also looks good on Twitter).

Top: month name in serif, large ("■■■ 2026")

Middle: a centered visual element — the four-category bar in restrained colors, OR a brush-stroke savings indicator (a circle filled to the savings %)

Bottom: a single line of text — the user's response to question 4 (the improvement intent), if they opted in

Bottom-right corner: tiny "digitalkakeibo.app" watermark

Texture: subtle washi paper background (low-opacity texture asset)

Generation approach: html2canvas

Build the card as a hidden React component, then convert to image via html2canvas. Cleaner than Canvas API for complex layouts.

```
import html2canvas from 'html2canvas';

async function generateCard(monthData) {
  const node = document.getElementById('share-card-template');
  const canvas = await html2canvas(node, {
    width: 1080,
    height: 1920,
    scale: 2, // retina
    backgroundColor: null
  });
  return canvas.toDataURL('image/png');
}
```

Privacy controls

Users see the card preview before sharing. Two toggles:

"Show amounts" (default: off — most users want privacy)

"Show my reflection" (default: off — reflections are sacred)

Share flow

Use the Web Share API where available:

```
if (navigator.share) {  
  await navigator.share({  
    title: 'My Kakeibo - May 2026',  
    files: [new File([blob], 'kakeibo.png', { type: 'image/png' })]  
  });  
}
```

The /share/:cardId landing page

When users post to Twitter, they often paste a link. Each generated card can have an optional public URL with proper Open Graph tags. This makes Twitter unfurls show the beautiful card preview.

// Next.js metadata for /share/:cardId

```
export const metadata = {  
  openGraph: {  
    images: [`/api/og/card/${cardId}.png`]`  
  }  
};
```

Acceptance criteria

Generated card looks indistinguishable from a designer's work

Card generation completes in under 3 seconds

Card looks good on Instagram, Twitter, Pinterest

Privacy toggles work and respect user's choice

Share flow uses native share sheet on mobile

Watermark is present but tasteful (not corporate)

Common pitfalls

html2canvas can be quirky with custom fonts — preload them and wait before rendering

Test on real iPhones — Web Share API behavior differs

Keep the watermark elegant. "Made with digitalkakeibo.app" in a thin font, not a logo splash.

What you're building

Most users will quit in the first 60 seconds if they don't understand the product. For Digital Kakeibo, you have a harder problem — you need to teach them the kakeibo philosophy, not just the app. Make this fast, beautiful, and respectful.

Features built

Onboarding flow (4-5 screens, swipeable)

Brief kakeibo history mini-lesson

Currency + locale selection

First entry creation as part of onboarding

Welcome email

Empty-state coaching throughout the app

Technical deep-dive

Onboarding sequence

Welcome screen — "■■■■ — Mindful money since 1904"

Brief story — single screen about Hani Motoko, with one stunning illustration or photo

The four categories explained — short, beautiful explanations

Settings — locale (English/Japanese), currency (JPY/USD/EUR/GBP), reflection day

First entry — guided: "Try it now. Record one expense from this week."

Implementation

Use a single OnboardingFlow component with internal step state. Persist completion to user record:

// User schema addition

onboardingCompleted: { type: Boolean, default: false },

onboardingCompletedAt: Date

Welcome email

Send via Resend. Plain HTML, brand-aligned. Three short paragraphs:

Welcome — thank user by first name

The practice — one paragraph on what to expect (weekly reflection on Sundays, monthly close on the 1st)

A quote from Hani Motoko or about kakeibo

Empty states everywhere

Every screen needs a thoughtful empty state:

Dashboard with no entries → "Your first step"

Reflection history empty → "Your reflections will appear here on Sundays"

Monthly archive empty → "Your kakeibo book begins this month"

Acceptance criteria

Onboarding takes ≤ 90 seconds for a determined user

Skip option exists but is small (advanced users only)

First entry created during onboarding appears on dashboard immediately

Welcome email arrives within 1 minute of signup

Returning to onboarding mid-flow resumes correctly

Locale selection persists and changes UI language immediately

Common pitfalls

Don't make it 10 screens — 5 is the maximum

Don't show progress dots — let it feel like a story, not a setup wizard

Don't ask for too much info upfront — only what's needed

Test the skip path — users skipping should still have a working app

What you're building

This is the phase that separates portfolio projects from products. Polish is invisible when done right and obvious when skipped. Spend a week here — recruiters can tell the difference within 30 seconds of using the app.

Features built

Loading states for every async operation

Error states with recovery actions

Page transitions and motion polish

Accessibility audit (WCAG 2.1 AA)

Performance optimization (Lighthouse 95+)

PWA setup (installable, offline-capable for viewing)

SEO and OpenGraph tags for landing/share pages

Technical deep-dive

Motion polish

Use Framer Motion for transitions. Keep them slow and gentle (250-400ms). The brand is restraint — no bouncy springs. Default ease is 'easeOut' or a custom cubic-bezier.

```
const slowEase = [0.16, 1, 0.3, 1]; // tailwind's ease-out-quad
```

```
<motion.div
```

```
  initial={{ opacity: 0, y: 8 }}
```

```
  animate={{ opacity: 1, y: 0 }}
```

```
  transition={{ duration: 0.4, ease: slowEase }}
```

```
>
```

Accessibility checklist

All interactive elements reachable via keyboard

Focus rings visible (don't hide them — restyle them)

ARIA labels on icon-only buttons

Color contrast 4.5:1 minimum for text

Screen reader tested for entry flow and reflection flow

`prefers-reduced-motion` respected — skip animations if user prefers

Performance targets

Lighthouse Performance: 95+

Time to Interactive: under 2s on 4G

First Contentful Paint: under 1s

Bundle size: under 200KB JS gzipped

Optimization techniques

Use Next.js Image component for any imagery

Dynamic imports for heavy components (html2canvas, charts)

Server components where possible (Next.js 14 App Router)

Cache aggregation queries with React Query

Index MongoDB queries (already done in earlier phases)

PWA configuration

Add a service worker via next-pwa. Cache the app shell + read-only views (past reflections viewable offline). Don't try offline write — sync conflicts get ugly.

Error boundaries

```
// app/error.tsx
```

```
'use client';
```

```
export default function Error({ error, reset }) {
```

```
  return (
```

```
    <div className='...'>
```

```
      <h2>Something went quietly wrong.</h2>
```

```
      <button onClick={reset}>Try again</button>
```

```
    </div>
```

```
  );
```

```
}
```

Acceptance criteria

Lighthouse scores: 95+ Performance, 100 Accessibility, 100 Best Practices, 100 SEO

Every async action has a loading and error state

Tab through the entire app with keyboard — every action is reachable

App installable as PWA on iOS and Android

Sharing the URL on Twitter shows a beautiful preview

Tested on iPhone SE, iPhone 14 Pro, Android (Pixel), and desktop Chrome/Safari

Common pitfalls

Don't add features in this phase — only polish what exists

Real device testing reveals issues simulators hide

Bundle analyzer (next-bundle-analyzer) catches accidental large imports

What you're building

The phase most developers skip — and the reason their portfolio projects have zero users. This phase is not optional. A built-but-unlaunched product is a school project. A launched product with users is a job application that hires itself.

Features built / activities completed

Marketing landing page (separate from app — story-driven)

Product Hunt launch

Reddit launch posts (carefully crafted, non-spammy)

Twitter/X founder thread

Show HN post

Outreach to 10 kakeibo bloggers/writers

Analytics in place (Plausible, no creepy tracking)

Feedback mechanism (in-app + email)

Marketing assets to prepare

The landing page

Single page, scrolling sections. Every word is intentional.

Hero: "■■■■ — Mindful money since 1904." Subhead: "A digital kakeibo, faithful to the practice."

The story (3 short paragraphs about Hani Motoko and why this app exists)

How it works — three sequential phone mockups

The four categories — illustrated

A real Monthly Card (her own, used as proof)

Quotes from beta testers

Single CTA: "Begin"

Product Hunt strategy

Launch day: Tuesday (less competitive than Wednesday)

Have 5+ supporters lined up (friends, beta users) to upvote and comment in the first hour

First comment from her: the founder story (NOT a feature list)

Respond to every comment personally within 30 minutes

Reddit posts

Subreddits to target (in order of priority): r/kakeibo, r/japanlife, r/personalfinance, r/japanesefood, r/minimalism, r/MinimalLifestyle

Title formulas that work:

"I built a digital kakeibo that respects the actual 1904 method (not just the name)"

"After 6 months of paper kakeibo, I built the digital version I wished existed"

"Why every 'kakeibo app' is wrong (and what I built instead)"

Founder thread on Twitter/X

Format: 8-12 tweets. Each tweet a single image or short text. The story is:

Hook: "I built an app that has no automation. On purpose."

Background: kakeibo's history (with the photo of Hani Motoko)

The problem: existing apps and what they get wrong

The four categories

The four questions

A monthly card screenshot

Beta user quote

Link + CTA

Analytics

Plausible Analytics on landing page and app. Track:

Signup conversion rate

First entry completion (within 24 hours of signup)

Weekly reflection completion rate (THE KEY METRIC)

Monthly ritual completion rate

Cards generated

Cards shared

Feedback loop

In-app: small "feedback" link in settings — opens an email pre-filled to her address

Post-launch: personally email the first 100 signups asking what worked, what didn't

Acceptance criteria

App is launched on Product Hunt

Posted to Reddit (3+ subs) without being banned for spam

Twitter thread has 1000+ impressions

100+ signups in launch week

Real users have completed at least one weekly reflection

She has a list of the next 20 things to fix or add (good problem to have)

Common pitfalls

Don't post in subreddits without participating beforehand — mods will smell promotional

Don't ignore the first 100 signups — they are the seed audience

Done is better than perfect.